Bulkhead Pattern

watertight compartments for your resources

Watch the 4-min video

In one line: give each dependency, workload or tenant its own bounded pool of threads, connections or instances, so when one of them slows down or floods, it exhausts only its own compartment and everything else keeps working.

1 One leak shouldn't sink the ship

A ship's hull is split into watertight compartments, called bulkheads: if one floods, the others stay dry. Most services have one big shared compartment: one thread pool, one connection pool, one cluster for every customer. Then a single slow dependency, or a single noisy customer, can drown all of it.

2 How a shared pool spreads failure

A service has **100 threads** and calls inventory, pricing and reviews. Inventory slows from 50 ms to 5 s. Its calls hold threads 100× longer, until all 100 are waiting on inventory. Calls to the two healthy dependencies can't get a thread either: a failure in one place has spread everywhere. Split the pool into **40 / 30 / 30** and the slow dependency can fill only its 40; extra calls to it are rejected immediately, and pricing and reviews keep working.

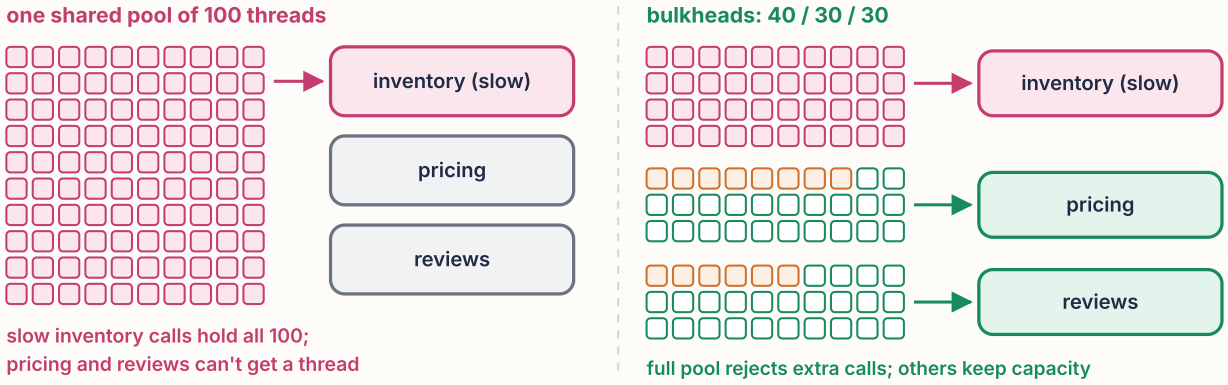


Figure 1. Left: one slow dependency occupies the shared pool. Right: it fills only its own compartment.

3 Bulkheads at every level

Level	What is isolated	Example
In-process	thread pool or semaphore per dependency	Resilience4j `Bulkhead` with max 40 concurrent calls
Connections	connection pool per database or downstream	reporting queries get their own pool, so they can't starve checkout
Instances	separate deployments or clusters per workload	critical API on one node pool, batch jobs on another
Tenants	capacity or concurrency per customer	per-tenant limits and queues; whole cells per group (lesson 16)

The goal is a small **blast radius**: how much of the system one failure can affect. The smaller the compartments, the smaller the radius. **Cell-based architecture** (lesson 16) takes this to the extreme: independent copies of the whole stack, each serving a slice of customers.

WORKED EXAMPLE: PREMIUM VS FREE TENANT POOLS

A report service shared one worker pool. A free-tier user submitted thousands of huge reports, and paying customers waited hours. Now premium tenants get a **dedicated pool of 30 workers**, free tenants share a **pool of 10**, and each tenant may run at most **2 jobs at once**. A flood of free jobs only lengthens the free queue, and even there one tenant can hold just 2 of the 10 workers. Premium reports keep finishing on time.

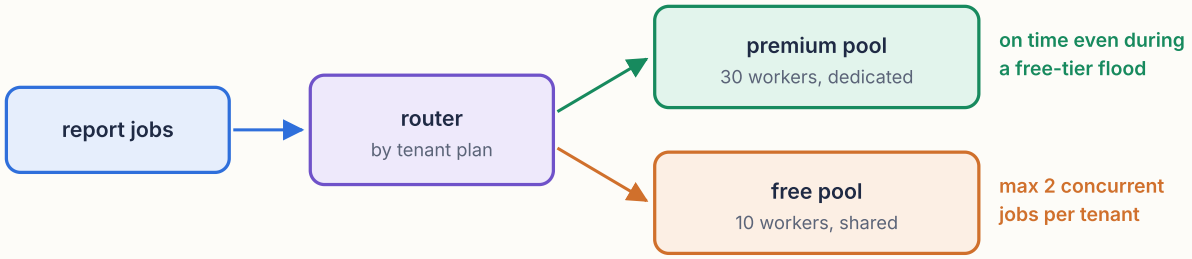


Figure 2. Two bulkheads: by plan (separate pools) and by tenant (a concurrency cap inside the shared pool).

COSTS AND PAIRINGS

- **Lower utilisation:** one pool may sit idle while another is full; that spare capacity is the price of isolation.
- **Sizing:** size each pool from measured load (Little's law: rate × latency, plus headroom) and revisit it.
- **Pair with timeouts** (lesson 4) that free threads, and **circuit breakers** (lesson 6) that stop calls to a dead dependency; the bulkhead caps the damage meanwhile.
- **Rejected calls** need a fallback or a clear error (lesson 8).

4 Key terms

Bulkhead: isolated, bounded pool of resources

Blast radius: how much of the system one failure can affect

Semaphore bulkhead: cap on concurrent calls, no separate threads

Thread-pool bulkhead: dedicated threads per dependency

Noisy neighbour: tenant whose load hurts others on shared resources

Cell: independent full copy of the stack for a slice of users

5 Interview questions

Q1. What is a bulkhead? BASIC

Isolating resources (thread pools, connection pools, instances, capacity) per dependency or tenant, so a failure in one compartment can't exhaust resources everywhere.

Q2. Give an example. BASIC

Separate worker pools for premium and free tenants with per-tenant limits, or a dedicated thread pool per downstream dependency, so one slow dependency only fills its own pool.

Q3. Checkout and an analytics export share one database connection pool. What do you do? SCENARIO

Give each its own pool (or a read replica for analytics), sized from measured load, so a heavy export can't take the connections checkout needs.

Q4. One tenant's spike slows down everyone. How do you fix it? SCENARIO

Per-tenant concurrency limits or rate limits, separate queues, and for big tenants dedicated capacity or their own cell.

Q5. Why not just make the shared pool bigger? TRICKY

A slow dependency fills any size of pool eventually; a bigger pool only delays the outage and adds memory and downstream load. Isolation, not size, contains it.

Q6. What is the main cost of bulkheads? TRICKY

Partitioned capacity is used less efficiently: one pool idles while another rejects work, and each pool must be sized and tuned.

Next: SD-K-008 · Graceful Degradation backendroadmap.com